

DSAA3073: Theories in Data Science

Week 5: Cross-Validation & Model Selection Criteria

Concise Learning Sheet

Wei Wang · DSA, HKUST(GZ) · 2026-07-09

Explorative projection

Opening Challenge

The Hyperparameter Selection Dilemma

You've trained a ridge regression model on 100 samples and need to select the regularization parameter λ . You have three approaches:

1. **Validation set:** Split data 70/30, train on 70, validate on 30
2. **Leave-one-out CV:** Train 100 times, each time leaving out one sample
3. **AIC:** Use information criterion based on likelihood

Questions:

- Which method gives the most reliable estimate of test error?
- Which is computationally cheapest?
- Can you compute LOOCV without training 100 separate models?
- When would you prefer AIC over cross-validation?

This week formalizes these tradeoffs and reveals surprising computational shortcuts for linear models.

Introduction

The core challenge in statistical learning: we need to select hyperparameters and compare models without contaminating our test set. Training error is optimistically biased (underestimates true error), so we need honest estimates of generalization performance.

This week develops three complementary approaches:

1. **Cross-validation:** Empirical, distribution-free, works for any model
2. **Information criteria** (AIC/BIC): Parametric, theory-based, single fit
3. **Structural risk minimization:** Connects to VC theory from Week 2

Each trades off computational cost, statistical assumptions, and theoretical guarantees differently.

Cross-Validation Framework

The core problem: we need to select hyperparameters (like regularization strength λ) without contaminating our test set. How do we estimate test error honestly during the model development process?

? Student Question

“Why not just split into train/validation/test sets? Seems simple enough.”

A 70/30 train/validation split is straightforward: train on 70%, validate on 30%, pick the best hyperparameter, then test on a separate held-out set. What’s wrong with this approach?

The Validation Principle

Before diving into methods, let’s establish why we need validation at all. **Training error lies.** A complex model can perfectly fit training data (zero training error) while completely failing on test data.

Definition: Test Error Estimation

Given training data $S = \{(x_i, y_i)\}_{i=1}^n$, we want to estimate the **test error** (expected loss on new data) of the fitted model:

$$R_{\mathcal{D}}(\hat{f}) = \mathbb{E}_{x, y \sim \mathcal{D}} \left[(y - \hat{f}(x))^2 \right] \quad (1)$$

where $\hat{f}$ is trained on S and (x, y) is a fresh sample from $\mathcal{D}$.

Example: Training vs. Test Error

Fitting polynomials to $n = 20$ points from $f^*(x) = \sin(2\pi x) + \varepsilon$:

Degree 3: Training error = 0.12, Test error = 0.15 (close!)

Degree 15: Training error = 0.01 (excellent!), Test error = 0.45 (disaster!)

The degree-15 polynomial memorizes noise. We need an honest estimate.

The Natural Approach: Single Validation Set

“Just hold out 30% of data for validation. Train on 70%, validate on 30%. Pick the hyperparameter with lowest validation error. Simple!”

Definition: Validation Set Method

1. Split data: $S = S_{\text{train}} \cup S_{\text{val}}$ (e.g., 70/30 split)
2. Train on S_{train} : obtain fitted model $\hat{f}_{\text{train}}$
3. Evaluate on S_{val} :

$$\hat{R}_{\text{val}} = \frac{1}{|S_{\text{val}}|} \sum_{(x,y) \in S_{\text{val}}} \left(y - \hat{f}_{\text{train}(x)} \right)^2 \quad (2)$$

Advantages:

- Simple and fast (one model fit per candidate)
- Unbiased for the test error of model trained on S_{train}

Let's test this on a concrete problem.

The Variance Crisis: When Random Splits Matter

Example: Ridge Regression Hyperparameter Selection

Dataset: $n = 100$ samples, $d = 20$ features, comparing $\lambda \in \{0.01, 0.1, 1, 10, 100\}$

Method: 70/30 train/validation split

Random seed 42:

λ	Validation Error	Selected?
0.01	0.35	
0.1	0.32	
1.0	0.28	✓ minimum
10.0	0.34	
100.0	0.42	

Selected: $\lambda = 1.0$ with validation error 0.28

Random seed 123 (different split):

λ	Validation Error	Selected?
0.01	0.42	
0.1	0.28	✓ minimum
1.0	0.31	
10.0	0.38	
100.0	0.45	

Selected: $\lambda = 0.1$ with validation error 0.28

Random seed 999:

λ	Validation Error	Selected?
0.01	0.38	
0.1	0.42	
1.0	0.35	
10.0	0.31	
100.0	0.48	

Selected: $\lambda = 10.0$ with validation error 0.31

 Crisis**The Variance Crisis**

Three random splits of the same data selected **three different hyperparameters**:

- Seed 42: $\lambda = 1.0$
- Seed 123: $\lambda = 0.1$
- Seed 999: $\lambda = 10.0$

These differ by **100×**! Which is the “right” choice? The validation set approach gives **unstable estimates** because it depends heavily on which 30 samples ended up in the validation set.

Testing all three on fresh test data:

- $\lambda = 1.0$: true test error = 0.29
- $\lambda = 0.1$: true test error = 0.34
- $\lambda = 10.0$: true test error = 0.32

The “correct” choice ($\lambda = 1.0$) was selected only 1 out of 3 times!

Two problems with single validation set:

1. **High variance:** With only 30 validation samples, the estimate is noisy
 - Same λ gives validation errors ranging from 0.28 to 0.42 across splits
 - The best λ changes depending on which samples are held out
2. **Data waste:** Training on only 70 samples
 - Model trained on 70 samples has higher error than model trained on 100 samples
 - Validation error is pessimistically biased (overestimates true test error)

Numerical impact: Across 100 random seeds, the selected λ ranged from 0.01 to 100 (10,000× range!), and the validation error for the same λ had standard deviation 0.08—huge relative to mean error of 0.30.

The Key Insight: Reuse Data Efficiently **Aha Moment****The Resolution: Rotate the Validation Set**

Instead of one 70/30 split, use **multiple** splits and average the results!

Each sample gets to be:

- In the validation set exactly once (honest test)
- In the training set for all other splits (efficient data use)

Averaging over multiple splits reduces variance.

This is k-fold cross-validation.

k-Fold Cross-Validation

Definition: k-Fold Cross-Validation

1. Partition S into k disjoint **folds** (equal-sized subsets):

$$S = S_1 \cup \dots \cup S_k \text{ with } |S_i| \approx \frac{n}{k} \quad (3)$$

2. For $i = 1, \dots, k$:

- Train on $S \setminus S_i$ (all folds except fold i): obtain model $\hat{f}_{-i}$
- Validate on S_i (fold i): compute

$$\hat{R}_i = \frac{1}{|S_i|} \sum_{(x,y) \in S_i} (y - \hat{f}_{-i}(x))^2 \quad (4)$$

3. Average over all folds:

$$\text{CV}_k = \frac{1}{k} \sum_{i=1}^k \hat{R}_i = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}_{-i}(x_i))^2 \quad (5)$$

Each sample appears in validation exactly once.

Example: Same Data with 5-Fold CV

Dataset: same $n = 100$, $d = 20$, comparing same λ values

Method: 5-fold cross-validation

Random seed 42:

λ	CV Error	Std Dev	Selected?
0.01	0.35	0.04	
0.1	0.31	0.03	
1.0	0.29	0.03	✓ minimum
10.0	0.33	0.04	
100.0	0.43	0.05	

Selected: $\lambda = 1.0$ with CV error 0.29

Random seed 123:

λ	CV Error	Std Dev	Selected?
0.01	0.36	0.05	
0.1	0.32	0.04	
1.0	0.30	0.03	✓ minimum
10.0	0.34	0.04	
100.0	0.44	0.06	

Selected: $\lambda = 1.0$ with CV error 0.30

Random seed 999:

λ	CV Error	Std Dev	Selected?
0.01	0.34	0.04	
0.1	0.30	0.03	
1.0	0.28	0.03	✓ minimum
10.0	0.32	0.04	
100.0	0.42	0.05	

Selected: $\lambda = 1.0$ with CV error 0.28

Result: All three random seeds selected the **same** $\lambda = 1.0$! The CV error for this λ ranged only from 0.28 to 0.30 (much tighter than 0.28 to 0.42).

Comparison: Validation Set vs 5-Fold CV

Method	Selected λ	Estimate	True Test Error
Validation (seed 42)	1.0	0.28	0.29
Validation (seed 123)	0.1	0.28	0.34
Validation (seed 999)	10.0	0.31	0.32

5-fold CV (seed 42)	1.0	0.29	0.29
5-fold CV (seed 123)	1.0	0.30	0.29
5-fold CV (seed 999)	1.0	0.28	0.29

Why 5-fold CV is more stable:

1. More validation samples per estimate:

- Validation set: 30 samples (1 split)
- 5-fold CV: 100 samples total (20 per fold $\times$ 5 folds), averaged

2. Variance reduction through averaging:

- Validation set: single estimate, high variance
- 5-fold CV: average of 5 estimates, variance reduced by $\approx \sqrt{5} \approx 2.2\times$

3. Less pessimistic bias:

- Validation set: trains on 70 samples
- 5-fold CV: trains on 80 samples (closer to final 100)

The Bias-Variance Tradeoff in k

Key Takeaway

Choosing k in k-fold CV involves a fundamental tradeoff:

- **Small k** (e.g., $k = 5$):
 - Trains on less data ($\frac{k-1}{k}$ of samples) $\rightarrow$ **higher bias**
 - Training sets less correlated $\rightarrow$ **lower variance**
 - Faster (fewer model fits)
- **Large k** (e.g., $k = n$):
 - Trains on more data ($n - 1$ samples) $\rightarrow$ **lower bias**
 - Training sets highly overlapping $\rightarrow$ **higher variance**
 - Slower (more model fits)

Example: Variance vs k: Numerical Demonstration

Dataset: $n = 100$, evaluate for fixed $\lambda = 1.0$

k	Train Size	CV Error	Std Dev Across Seeds
3	67	0.31	0.05
5	80	0.29	0.03
10	90	0.28	0.06
20	95	0.27	0.08
100 (LOOCV)	99	0.27	0.00 (deterministic)

Observations:

- As k increases, CV error decreases (less biased, trains on more data)
- But standard deviation across random fold assignments increases (more variance)
- LOOCV is deterministic (no random splits) but we'll see it has high variance from a different source

Why does large k have higher variance?

The k validation errors are **correlated** because training sets overlap:

- With $k = 5$: Training sets share 75% of samples (60 out of 80)
- With $k = 100$ (LOOCV): Training sets share 98% of samples (98 out of 99)

When training sets overlap heavily, removing one sample barely changes the model, so validation errors are highly correlated. High correlation $\rightarrow$ high variance in the average.

Standard error of CV estimate:

$$\text{SE}(\text{CV}_k) \approx \frac{\sigma}{\sqrt{n}} \cdot \sqrt{1 + (n-1)\rho} \quad (6)$$

where ρ is the correlation between fold errors. For LOOCV, $\rho \approx 0.9-0.99$ (very high!), so:

$$\text{SE}(\text{CV}_n) \approx \frac{\sigma}{\sqrt{n}} \cdot \sqrt{1 + (n-1) \cdot 0.95} \approx \sigma \sqrt{n} \quad (7)$$

The standard error **grows** with n for LOOCV! Meanwhile, for $k = 5$, $\rho \approx 0.4-0.6$, giving much lower standard error.

Leave-One-Out Cross-Validation (LOOCV)

The extreme case: hold out only one sample at a time ($k = n$).

Definition: LOOCV

Special case of k-fold CV with $k = n$:

$$CV_n = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}_{-i}(x_i))^2 \quad (8)$$

where $\hat{f}_{-i}$ is trained on all data except (x_i, y_i) .

Properties:

- **Nearly unbiased:** Trains on $n - 1$ samples (almost full dataset)
- **Deterministic:** No random fold assignment (every sample gets its own fold)
- **High variance:** Training sets overlap by 98%, so errors highly correlated
- **Expensive:** Requires n model fits (seems prohibitive...)

Pause and Think

Question: Why is LOOCV nearly unbiased for test error?

Hint: Each $\hat{f}_{-i}$ trained on $n - 1$ samples. As $n \rightarrow \infty$, the difference between training on $n - 1$ vs n samples vanishes.

Answer:

LOOCV approximates the test error of the model trained on all n samples when the learning curve is smooth near n .

Comparison: k-fold CV choices

k	Bias	Variance	Computation	Typical Use
3-5	Moderate	Low	Fast	Default choice, small n
10	Low	Moderate	Medium	Standard, moderate n
n (LOOCV)	Very low	High	Slow	Need unbiased estimate

Common practice: Use $k = 5$ or $k = 10$ as default. The bias from training on 80-90% of data is typically small, and the variance reduction is substantial.

LOOCV: The Computational Challenge

LOOCV seems expensive: for $n = 1000$ samples, we need to fit 1000 separate models. For each hyperparameter candidate!

Naive computational cost:

- One model fit (e.g., ridge regression): $O(nd^2)$ operations
- LOOCV: n model fits $\rightarrow O(n^2d^2)$ operations
- Comparing 10 hyperparameters: $10 \times O(n^2d^2)$ operations

For $n = 1000, d = 50$:

- One fit: $1000 \times 50^2 = 2.5 \times 10^6$ operations

- Naive LOOCV: $1000 \times 2.5 \times 10^6 = 2.5 \times 10^9$ operations
- 10 hyperparameters: 2.5×10^{10} operations (25 billion!)

This seems prohibitively expensive. But for **linear models**, there's a remarkable shortcut...

Progress on backbone: LOOCV gives nearly unbiased estimates but requires n model fits. For $n = 1000$, this is 25 billion operations just to compare 10 hyperparameters. The computational cost seems to make LOOCV impractical for anything but tiny datasets.

But here's where we can exploit the special structure of linear models. When predictions are linear in the response vector ($\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$ for some matrix $\mathbf{H}$), the leave-one-out predictions can be computed from a **single** full-data fit using only the diagonal of $\mathbf{H}$. This is one of the most elegant computational tricks in statistical learning, reducing the cost from $O(n^2d^2)$ to $O(nd^2)$ —the same as fitting once.

The key insight: removing one observation from a linear model is a **rank-1 update** to the solution. The Sherman-Morrison formula tells us exactly how to adjust the predictions without refitting.

Efficient LOOCV for Linear Models

LOOCV trains on $n - 1$ samples and validates on the remaining one sample, repeated n times. This gives nearly unbiased estimates of test error. But there's a computational problem...

? Student Question

“LOOCV requires training n separate models. For $n=1000$, that's 1000 fits! Isn't this computationally prohibitive?”

With ridge regression on $n = 1000$ samples and $d = 50$ features, each model fit costs $O(nd^2)$ operations. Training 1000 models seems impossibly expensive. Can we really afford LOOCV?

The Naive Approach: Train n Times

Let's calculate the computational cost explicitly.

LOOCV definition (from Section 1):

$$CV_n = \frac{1}{n} \sum_{i=1}^n \left(y_i - \hat{f}_{-i}(x_i) \right)^2 \quad (9)$$

where $\hat{f}_{-i}$ is the model trained on all data except point (x_i, y_i) .

Naive implementation:

```
for i in range(n):
    # Remove point i
    X_train = X without row i
    y_train = y without entry i

    # Fit model on n-1 points
    model.fit(X_train, y_train) # Cost: O(n d^2)

    # Predict on held-out point i
    pred = model.predict(X[i])
    error[i] = (y[i] - pred)^2

cv_score = mean(error)
```

Computational cost per model fit:

- For linear regression (solving normal equations): $O(nd^2)$ operations
- For ridge regression: Also $O(nd^2)$ (matrix inversion)

Total cost for LOOCV:

- n model fits, each costing $O(nd^2)$
- Total: $O(n^2d^2)$ operations

The Computational Explosion

Example: Ridge Regression Hyperparameter Selection

Task: Compare 10 values of λ for ridge regression

Dataset: $n = 1000$ samples, $d = 50$ features

Cost per model fit:

$$nd^2 = 1000 \times 50^2 = 1000 \times 2500 = 2.5 \times 10^6 \text{ operations} \quad (10)$$

Cost for naive LOOCV (one λ value):

$$n \times (nd^2) = 1000 \times 2.5 \times 10^6 = 2.5 \times 10^9 \text{ operations} \quad (11)$$

Cost for 10 λ values:

$$10 \times 2.5 \times 10^9 = 2.5 \times 10^{10} \text{ operations} \quad (12)$$

25 billion operations just to select one hyperparameter!

 Crisis
The Computational Explosion

For $n = 1000, d = 50$:

- **Naive LOOCV:** 2,500,000,000 operations per λ
- **10 hyperparameters:** 25,000,000,000 operations total
- **50 billion operations** if we compare 20 λ values!

At 1 GFlop/s (1 billion floating point operations per second), this takes:

- One λ : 2.5 seconds
- 10 λ values: 25 seconds
- 20 λ values: 50 seconds

Seems manageable for $n=1000$, but what about larger datasets?

For $n = 10,000, d = 100$:

- Naive LOOCV per λ : $10^4 \times 10^4 \times 10^4 = 10^{12}$ operations
- 10 λ values: 10^{13} operations = **10 trillion operations**
- At 1 GFlop/s: **3 hours** just to try 10 hyperparameters!

This scales as $O(n^2d^2)$ —quadratic in sample size. For large datasets, LOOCV becomes intractable.

Comparison: LOOCV vs 5-fold CV computational cost

Method	Model Fits	Cost	Example ($n=1000, d=50$)
5-fold CV	5	$5 \times O(nd^2)$	1.25×10^7 ops
10-fold CV	10	$10 \times O(nd^2)$	2.5×10^7 ops
Naive LOOCV	n	$n \times O(nd^2) = O(n^2d^2)$	2.5×10^9 ops

The gap: LOOCV is $200\times$ **slower** than 5-fold CV for $n=1000$. This grows to $2000\times$ **slower** for $n=10,000$.

Why this matters:

LOOCV should be our **baseline** for model selection:

- Nearly unbiased (trains on $n-1$ samples)
- Deterministic (no random splits)
- Standard for comparing other methods

But if it's $200-2000\times$ more expensive than k-fold CV, we're forced to use k-fold, which has higher bias and variance. We're trading off statistical properties for computational feasibility.

Can we make LOOCV practical?

The Algebraic Insight: Linear Smoothers

Here's the key observation: for many models, predictions are **linear functions** of the response values.

Definition: Linear Smoother

A predictor $\hat{f}$ is a **linear smoother** if predictions are linear in $\mathbf{y}$:

$$\hat{\mathbf{y}} = \mathbf{H}\mathbf{y} \quad (13)$$

where $\mathbf{H} \in \mathbb{R}^{n \times n}$ is the **hat matrix** (also called **smoother matrix** or **influence matrix**). The matrix $\mathbf{H}$ depends only on the design matrix $\mathbf{X}$ and hyperparameters, **not** on the response values $\mathbf{y}$.

The name “hat matrix” comes from the fact that it “puts the hat on $\mathbf{y}$ ” to get predictions $\hat{\mathbf{y}}$.

Example: Common Linear Smoothers**Ordinary least squares:**

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \quad (14)$$

$$\hat{\mathbf{y}} = \mathbf{X} \hat{\beta} = \mathbf{X} (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} = \mathbf{H} \mathbf{y} \quad (15)$$

where $\mathbf{H} = \mathbf{X} (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$

Ridge regression:

$$\hat{\beta}_\lambda = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y} \quad (16)$$

$$\hat{\mathbf{y}} = \mathbf{X} \hat{\beta}_\lambda = \mathbf{X} (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y} = \mathbf{H}_\lambda \mathbf{y} \quad (17)$$

where $\mathbf{H}_\lambda = \mathbf{X} (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top$

k-nearest neighbors:

$$\hat{y}_i = \frac{1}{k} \sum_{j \in \mathcal{N}_k(x_i)} y_j \quad (18)$$

where $\mathcal{N}_k(x_i)$ are the k nearest neighbors of x_i .

In matrix form: $\hat{\mathbf{y}} = \mathbf{H} \mathbf{y}$ where

$$H_{ij} = \begin{cases} \frac{1}{k} & \text{if } x_j \in \mathcal{N}_k(x_i) \\ 0 & \text{otherwise} \end{cases} \quad (19)$$

Why this matters:

When predictions are linear in $\mathbf{y}$, removing one observation is a **rank-1 update** to the data matrix. There's a classical formula—the Sherman-Morrison formula—that tells us exactly how to adjust matrix inverses for rank-1 updates **without recomputing the entire inverse**.

This means we can compute what would happen if we removed point i **without actually refitting the model!**

 Aha Moment**The Resolution: Sherman-Morrison Formula**

For linear smoothers, removing one point is a rank-1 matrix update:

$$\mathbf{X}_{-i}^\top \mathbf{X}_{-i} = \mathbf{X}^\top \mathbf{X} - x_i x_i^\top \quad (20)$$

The Sherman-Morrison formula gives us the updated inverse $(\mathbf{X}_{-i}^\top \mathbf{X}_{-i})^{-1}$ directly from $(\mathbf{X}^\top \mathbf{X})^{-1}$ —**no refitting needed!**

This means the leave-one-out prediction $\hat{y}_{-i}(x_i)$ can be expressed algebraically in terms of the full-data fit $\hat{y}_i$ and the hat matrix diagonal H_{ii} .

Result: **Exact LOOCV from one model fit!**

The Exact Formula

Theorem: LOOCV for Linear Smoothers

For any linear smoother where $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$ with $\mathbf{H}$ depending only on $\mathbf{X}$ (not $\mathbf{y}$):

$$\text{CV}_n = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - H_{ii}} \right)^2 \quad (21)$$

where:

- $\hat{y}_i$ is the prediction at x_i using the **full dataset** (all n points)
- H_{ii} is the i -th diagonal element of the hat matrix $\mathbf{H}$

This formula is **mathematically exact**, not an approximation!

Proof sketch (OLS/Ridge):

Step 1: Relate leave-one-out prediction to full-data prediction.

By Sherman-Morrison formula (applied to the normal equations):

$$\hat{y}_{-i}(x_i) = \hat{y}_i - \frac{H_{ii}}{1 - H_{ii}}(y_i - \hat{y}_i) \quad (22)$$

Step 2: Rearrange to isolate the leave-one-out error.

$$y_i - \hat{y}_{-i}(x_i) = y_i - \hat{y}_i + \frac{H_{ii}}{1 - H_{ii}}(y_i - \hat{y}_i) \quad (23)$$

$$= (y_i - \hat{y}_i) \left(1 + \frac{H_{ii}}{1 - H_{ii}} \right) \quad (24)$$

$$= (y_i - \hat{y}_i) \frac{1 - H_{ii} + H_{ii}}{1 - H_{ii}} \quad (25)$$

$$= \frac{y_i - \hat{y}_i}{1 - H_{ii}} \quad (26)$$

Step 3: Square and average.

$$\text{CV}_n = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_{-i}(x_i))^2 = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - H_{ii}} \right)^2 \quad (27)$$

Key Takeaway**Computational savings:****Before (naive LOOCV):**

- Fit n separate models: $O(n^2 d^2)$ operations
- For $n = 1000, d = 50$: 2.5 billion operations

After (shortcut formula):

1. Fit model once: $O(nd^2) = 2.5$ million operations
2. Compute hat matrix diagonal: $O(nd^2) = 2.5$ million operations
3. Apply LOOCV formula: $O(n) = 1,000$ operations

Total: $O(nd^2) \approx 5$ million operations

Speedup: 500× for $n=1000$, and it gets better as n grows!

Example: Numerical Verification: Exact, Not Approximate

Dataset: $n = 20$ samples, $d = 5$ features, ridge regression with $\lambda = 1.0$

Method 1: Naive LOOCV (refit 20 times)

```
errors = []
for i in range(20):
    X_train = X without row i
    y_train = y without entry i
    model.fit(X_train, y_train)
    pred = model.predict(X[i])
    errors.append((y[i] - pred)2)
cv_naive = mean(errors)
```

Result: $cv_naive = 0.437162$

Method 2: Shortcut formula (fit once)

```
model.fit(X, y)
y_pred = model.predict(X)
H_diag = compute_hat_diagonal(X, lambda=1.0)

adjusted_errors = [(y[i] - y_pred[i])2 / (1 - H_diag[i])2
                    for i in range(20)]
cv_shortcut = mean(adjusted_errors)
```

Result: $cv_shortcut = 0.437162$

Difference: 0.000000 (identical to machine precision!)

This isn't an approximation—it's the **exact** same value. The Sherman-Morrison formula is mathematically rigorous.

Interpretation: Leverage and Influence

The diagonal elements H_{ii} of the hat matrix have a beautiful interpretation.

Definition: Leverage

The **leverage** of point i is H_{ii} , the i -th diagonal element of the hat matrix.

Geometric interpretation: H_{ii} measures how far x_i is from the center of the other points in feature space.

Properties:

- $0 \leq H_{ii} \leq 1$ always
- $\sum_{i=1}^n H_{ii} = \text{tr}(\mathbf{H}) = d$ (total leverage equals degrees of freedom)
- Average leverage: $\bar{H} = \frac{d}{n}$

Why “leverage”?

Points with high H_{ii} have **strong influence** on the fit—they “pull” the regression surface toward them. Removing such a point changes the fit substantially.

Example: Leverage in Simple Linear Regression

Dataset: $n = 11$ points

- 10 points at $x \in \{0, 1, 2, \dots, 9\}$ with $y = 2x + \varepsilon$
- 1 point at $x = 50$ with $y = 100$

Hat matrix diagonal:

- Points at $x = 0, 1, \dots, 9$: $H_{ii} \approx 0.1$ (low leverage)
- Point at $x = 50$: $H_{11,11} \approx 0.92$ (very high leverage!)

Interpretation:

- Regular points: Close to other points, low leverage, little individual influence
- Outlying point: Far from others, high leverage, dominates the fit

LOOCV formula impact:

For regular point with residual $y_i - \hat{y}_i = 0.3$:

$$\text{Contribution: } \left(\frac{0.3}{1 - 0.1} \right)^2 = \left(\frac{0.3}{0.9} \right)^2 = 0.11 \quad (28)$$

For high-leverage point with same residual 0.3:

$$\text{Contribution: } \left(\frac{0.3}{1 - 0.92} \right)^2 = \left(\frac{0.3}{0.08} \right)^2 = 14.06 \quad (29)$$

The high-leverage point contributes 128× more to LOOCV!

This makes sense: removing this influential point drastically changes the fit, so its leave-one-out error is much larger.

The denominator $(1 - H_{ii})$ as inflation factor:

Leverage H_{ii}	Inflation $\frac{1}{1-H_{ii}}$	Interpretation	Typical For
0.1	$1.11\times$	Low influence	Regular points

0.3	1.43×	Moderate influence	Slightly unusual points
0.5	2.00×	High influence	Boundary points
0.8	5.00×	Very high influence	Outliers in X-space
0.95	20.00×	Extreme influence	Far outliers

As $H_{ii} \rightarrow 1$, the inflation factor explodes. This captures the intuition that removing a perfect-leverage point (which the model interpolates exactly) would completely change the leave-one-out prediction.

Pause and Think

Question: What does it mean if $H_{ii} = 1$?

Hint: The model interpolates point i exactly: $\hat{y}_i = y_i$ always, regardless of the response value.

Answer:

With $H_{ii} = 1$, the denominator $(1 - H_{ii}) = 0$, making LOOCV undefined (division by zero). This happens when point i has “perfect leverage”—removing it makes the leave-one-out prediction undefined. In practice, regularization ($\lambda > 0$) ensures $H_{ii} < 1$.

Scope: When Does the Shortcut Work?

The LOOCV shortcut requires **linear smoothers**—models where $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$ with $\mathbf{H}$ depending only on $\mathbf{X}$.

Works for:

- Ordinary least squares
- Ridge regression
- Kernel ridge regression
- Linear smoothing splines
- k-nearest neighbors (k-NN)

Does NOT work for:

- Neural networks
- Decision trees
- Random forests
- Support vector machines (nonlinear kernels)
- Gradient boosting

Example: Why Neural Networks Don't Qualify

Neural network with one hidden layer:

$$\hat{y}_i = \sum_{j=1}^h w_j \sigma(v_j^\top x_i) \quad (30)$$

where $\mathbf{w}, \mathbf{v}$ are learned parameters and σ is a nonlinear activation.

Problem: The parameters $\mathbf{w}, \mathbf{v}$ are fit by minimizing loss on $\mathbf{y}$:

$$(\mathbf{w}, \mathbf{v}) = \operatorname{argmin}_{\mathbf{w}, \mathbf{v}} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (31)$$

When you remove point (x_j, y_j) , the optimization landscape changes **nonlinearly**. The new parameters $(\mathbf{w}_{-j}, \mathbf{v}_{-j})$ can't be expressed as linear functions of $\mathbf{y}$.

You can't write $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$ for a fixed $\mathbf{H}$ independent of $\mathbf{y}$.

Must refit n times—no shortcut available.

Summary: Linear smoother checklist

Property	Required?	Verification
Predictions linear in $\mathbf{y}$	✓ Yes	Check if $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$
$\mathbf{H}$ depends only on $\mathbf{X}$	✓ Yes	Verify $\mathbf{H}$ not a function of $\mathbf{y}$
$H_{ii} < 1$ for all i	✓ Yes	Add regularization if needed
Closed-form solution	Not required	Can use numerical $\mathbf{H}$ if no closed form

Summary: Making LOOCV Practical**Key Takeaway**

The breakthrough: For linear smoothers, LOOCV reduces from $O(n^2 d^2)$ to $O(nd^2)$ —a **factor of n speedup**.

Numerical impact:

- $n = 1000$: Naive 2.5 billion ops → Shortcut 5 million ops (**500× faster**)
- $n = 10,000$: Naive 1 trillion ops → Shortcut 200 million ops (**5,000× faster**)

This makes LOOCV **competitive with k-fold CV** for linear models, while maintaining near-zero bias.

The formula:

$$\text{CV}_n = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - H_{ii}} \right)^2 \quad (32)$$

is not an approximation—it's **mathematically exact** for any linear smoother.

When to use LOOCV shortcut:

- Linear models (OLS, ridge, kernel methods, k-NN)
- Need nearly unbiased test error estimate
- Computational cost same as one model fit

When NOT to use:

- Nonlinear models (neural networks, trees, SVMs)—must use k-fold CV instead
- High-leverage points cause instability—consider GCV

Progress on backbone: We've solved the computational problem for empirical cross-validation. But we still need to fit the model at least once, and for nonlinear models, we need k fits. Can we estimate test error from a **single fit** without any resampling at all? This brings us to information criteria.

Information-Theoretic Criteria

Cross-validation estimates test error by refitting models on different data subsets. Even with the LOOCV shortcut for linear models, we still need at least one full model fit. For nonlinear models, we need k fits.

? Student Question

“Can we estimate test error from a single model fit without any data splitting or refitting?”

Cross-validation is computationally expensive for nonlinear models and complex hyperparameter searches. Is there a theoretical shortcut that works from just the training likelihood?

The Direct Approach: Use Training Likelihood

For parametric models, we can compute the likelihood of the observed data given the fitted parameters. Why not use this as a proxy for test performance?

Definition: Training Log-Likelihood

For a parametric model with parameters θ , the **training log-likelihood** is:

$$\log L(\hat{\theta}) = \sum_{i=1}^n \log p(y_i | x_i, \hat{\theta}) \quad (33)$$

where $\hat{\theta}$ maximizes the likelihood on the training data (MLE).

For Gaussian regression: $\log L(\hat{\theta}) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \text{RSS}$

“Just pick the model with highest training likelihood! More likely data = better model.”

Example: Polynomial Regression Likelihood

Dataset: $n = 50$ points from $f^*(x) = \sin(2\pi x) + \varepsilon$, comparing polynomial degrees

For Gaussian noise: $-2 \log L(\hat{\theta}) = n \log(\frac{\text{RSS}}{n}) + \text{const}$ (related to RSS)

Degree	Training $-2 \log L$	Test $-2 \log L$
1	-95	-98
2	-100	-105
3	-110	-112
5	-125	-118
8	-145	-105
12	-165	-85

Naive approach: Pick degree 12 (lowest training log-likelihood = -165)

Reality: Degree 12 has **terrible** test log-likelihood = -85 (80 log-likelihood units worse!)

The Optimism Crisis**! Crisis****Training Likelihood Optimism**

For degree 12 polynomial ($n = 50$):

- Training log-likelihood: **-165** (excellent!)
- Test log-likelihood: **-85** (disaster!)
- **Gap: 80 log-likelihood units**

The training likelihood is optimistically biased because $\hat{\theta}$ was specifically chosen to maximize fit to the training data.

As model complexity increases, this bias grows:

- Degree 2: gap = 5 units (small)
- Degree 5: gap = 7 units (moderate)
- Degree 8: gap = 40 units (large!)
- Degree 12: gap = 80 units (enormous!)

Training likelihood systematically overestimates model quality for complex models.

Why does this happen?

The parameters $\hat{\theta}$ are chosen to maximize

$$\sum_{i=1}^n \log p(y_i \mid x_i, \theta) \quad (34)$$

. This optimization “uses up” information from the training set—the model gets to “see” the noise in the training data and adapt to it.

When evaluated on fresh test data, this adaptation doesn’t help (it’s fitting noise, not signal), so test likelihood is lower.

The bias depends on model complexity:

- More parameters → more ability to fit noise → larger optimism

The Asymptotic Correction: AIC and BIC

Under certain regularity conditions, asymptotic theory tells us the expected optimism equals the number of parameters.

Aha Moment

The Resolution: Penalize by Model Complexity

Akaike’s Information Criterion (AIC) corrects training likelihood by adding a penalty proportional to the number of parameters:

$$\text{AIC} = -2 \log L(\hat{\theta}) + 2k \quad (35)$$

Why exactly $2k$? Asymptotic theory reveals that when you fit k parameters by maximum likelihood, the training log-likelihood is optimistically biased upward by approximately k in expectation. Since we use $-2 \log L$ (for historical reasons related to chi-squared distributions), the correction becomes $+2k$.

This isn’t arbitrary—it comes from analyzing how the fitted parameters “use up” degrees of freedom to fit noise in the training data. Each parameter costs you one degree of freedom on average, inflating goodness-of-fit by 1 unit of log-likelihood.

Definition: Information Criteria for Model Selection

Two closely related criteria for parametric model selection:

Criterion	Formula	Penalty	Goal
AIC	$-2 \log L(\hat{\theta}) + 2k$	$2k$	Minimize prediction error
BIC	$-2 \log L(\hat{\theta}) + k \log n$	$k \log n$	Find true model

where:

- $\log L(\hat{\theta})$ is the maximized log-likelihood on training data
- k is the number of fitted parameters
- n is the sample size
- Lower values indicate better models

Key difference: BIC's penalty $k \log n$ grows with sample size n , making it more conservative for large datasets. AIC's fixed penalty $2k$ is prediction-oriented.

When to use each:

- **AIC:** Goal is prediction/forecasting, small to moderate n
- **BIC:** Goal is inference/finding true model structure, large n

Crossover: For $n < \exp(2) \approx 7.4$, BIC is more liberal than AIC (since $\log n < 2$). For $n > 7.4$, BIC becomes increasingly conservative.

Example: AIC/BIC Polynomial Selection

Same dataset: $n = 50$, polynomials fit to sine function

Degree	k	$-2 \log L$	AIC	BIC	True Test $-2 \log L$
1	2	-95	-91	-89	-98
2	3	-100	-94	-91	-105
3	4	-110	-102	-98	-112
5	6	-125	-113	-107	-118
8	9	-145	-127	-117	-105
12	13	-165	-139	-126	-85

AIC selects: Degree 3 (minimum AIC = -102) **BIC selects:** Degree 2 (minimum BIC = -91, more conservative) **True best:** Degree 3 (test = -112)

AIC correctly identifies degree 3! The penalty prevented overfitting to degree 8 or 12.

AIC vs BIC tradeoffs:

Criterion	Penalty	Goal	Best For
AIC	$2k$	Minimize prediction error	Forecasting, small n
BIC	$k \log n$	Find true model	Model identification, large n

Key insight: For small n , BIC penalty $k \log n$ can be too conservative. For $n < 8$, even $k \log n < 2k$, so BIC always more conservative than AIC.

Crossover at $n \approx \exp(2) \approx 7.4$, where $\log n = 2$.

For practical model selection:

- $n < 100$: Prefer AIC (BIC may underfit)
- $n > 1000$: Both work, BIC more parsimonious
- Goal is prediction: Use AIC
- Goal is inference (identify true model): Use BIC

Scope: When Do AIC/BIC Apply?

Information criteria require specific assumptions:

Required:

1. **Parametric model** with finite-dimensional parameters $\theta \in \mathbb{R}^k$
2. **Maximum likelihood estimation** (or asymptotically equivalent)
3. **Regularity conditions** (smooth log-likelihood, positive-definite Fisher information)

Works for:

- Linear regression (Gaussian likelihood)
- Generalized linear models (logistic, Poisson regression)
- Time series models (ARIMA, GARCH)
- Parametric survival models

Does NOT work for:

- Tree ensembles (random forests, gradient boosting)—parameter count ambiguous
- k-nearest neighbors—not parametric
- Support vector machines—not MLE-based
- Neural networks—regularity conditions often violated

For non-parametric or algorithmic models: **must use cross-validation**.

Connection to Test Error

AIC and BIC estimate different quantities:

AIC: Estimates expected Kullback-Leibler divergence from true distribution to fitted model

- Minimizing AIC $\approx$ minimizing expected test log-loss
- Prediction-oriented

BIC: Approximates Bayesian posterior probability of model

- Consistent: selects true model as $n \rightarrow \infty$ (if true model is in candidate set)
- Inference-oriented

Neither directly estimates squared error! But for Gaussian models:

$$-2 \log L(\hat{\theta}) \approx n \log(\text{RSS}) + \text{const} \quad (36)$$

So minimizing AIC/BIC $\approx$ minimizing RSS-based criteria, which relates to squared error.

Summary: Information criteria provide theory-based shortcuts

Key Takeaway

The advantage: Single model fit, no data splitting

The cost: Requires parametric model with MLE and regularity conditions

When to use:

- Parametric models (GLMs, time series)
- Comparing non-nested models
- Large number of candidates (cheaper than CV)

When NOT to use:

- Non-parametric methods (trees, k-NN)
- Neural networks (regularity conditions questionable)
- Small samples where asymptotic theory doesn't hold

For models where AIC/BIC don't apply: **use cross-validation.**

Progress on backbone: We now have three approaches: empirical (CV), parametric efficient (AIC/BIC), and we're about to see how these connect to the distribution-free VC theory from Week 2 through Structural Risk Minimization.

Practical Considerations

We now have multiple methods for model selection: validation set, k-fold CV, LOOCV, AIC, BIC. Which should you use for your problem?

? Student Question

“With so many methods, how do I choose? They all estimate test error—which is ‘best’?”

Each method has different assumptions, computational costs, and statistical properties. The “best” method depends on your context.

Method Comparison

Decision Framework

Step 1: Check model type

- **Parametric (GLM, ARIMA)?** → Consider AIC/BIC
- **Linear smoother (OLS, ridge, k-NN)?** → LOOCV shortcut available
- **Nonlinear (neural net, trees)?** → Must use k-fold CV

Step 2: Consider sample size

- **$n < 100$:** Use 5-fold CV or LOOCV (don't waste data)
- **$100 < n < 1000$:** Use 10-fold CV or LOOCV (if linear)
- **$n > 1000$:** Use 5-fold CV or validation set (variance less critical)

Step 3: Consider number of candidates

- **< 5 candidates:** Any method fine
- **5-20 candidates:** k-fold CV or AIC/BIC
- **> 20 candidates:** Need nested CV to avoid selection bias

Step 4: Consider goal

- **Prediction:** AIC or CV
- **Inference (find true model):** BIC
- **Uncertainty quantification:** CV (can compute std error)

Step 5: When selecting many hyperparameters (>20 combinations)

When comparing many models, use **nested CV** to avoid selection bias:

- **Outer loop:** Estimates true performance (unbiased)
- **Inner loop:** Selects hyperparameters (biased, but isolated from outer estimate)

Example: Nested CV

Comparing 50 hyperparameter combinations:

Wrong (single-level CV):

```
for λ in candidates:
    cv_score[λ] = 5-fold CV score
best_λ = argmin(cv_score)
report: cv_score[best_λ] # Optimistically biased!
```

Correct (nested CV):

```
for outer_fold in 1..5: # Outer: performance estimation
    train_outer, val_outer = split
    for λ in candidates: # Inner: hyperparameter selection
        cv_inner[λ] = 5-fold CV on train_outer
    best_λ_fold = argmin(cv_inner)
    train model with best_λ_fold on train_outer
    error_outer[fold] = evaluate on val_outer
report: mean(error_outer) # Unbiased!
```

Method	Assumptions	Bias	Variance	Cost (per λ)	Best For
Validation set	None	High	High	$O(nd^2)$	Very large n
5-fold CV	None	Moderate	Low	$5 \times O(nd^2)$	Default choice
10-fold CV	None	Low	Moderate	$10 \times O(nd^2)$	Standard
LOOCV	None	Very low	High	$O(n^2d^2)$ naive	Need unbiased
LOOCV (linear)	Linear smoother	Very low	High	$O(nd^2)$	Linear models
AIC	Parametric MLE	Low	Low	$O(nd^2)$	Parametric, prediction
BIC	Parametric MLE	Low	Low	$O(nd^2)$	Parametric, inference

Common Pitfalls

Pitfall 1: Selection bias with validation set reuse

⚠ Crisis

The Selection Bias Trap

You try 20 hyperparameter values, pick the one with lowest validation error (say, 0.22), and report “my model achieves 0.22 test error.”

Wrong! You tried 20 values and picked the best—that’s selection bias. True test error on fresh data: 0.27 (higher).

Solution: When tuning many hyperparameters, use nested CV (see Decision Framework, Step 5) or hold out a final test set.

Pitfall 2: Scope violations

Don’t use AIC/BIC for non-parametric methods!

Model	AIC/BIC Valid?	Use Instead
Linear regression	✓ Yes	Can also use LOOCV shortcut
Logistic regression	✓ Yes	AIC/BIC or CV
Random forest	✗ No	CV only (parameter count unclear)
k-NN	✗ No	LOOCV shortcut works (linear smoother)
Neural network	✗ No	CV only (regularity conditions fail)

Computational Complexity Summary

Method	Fits per λ	Cost per λ	Total (20 λ values)
Validation set	1	$O(nd^2)$	$20 \times O(nd^2)$
5-fold CV	5	$5 \times O(nd^2)$	$100 \times O(nd^2)$
LOOCV (naive)	n	$n \times O(nd^2)$	$20n \times O(nd^2)$
LOOCV (shortcut)	1	$O(nd^2)$	$20 \times O(nd^2)$
AIC/BIC	1	$O(nd^2)$	$20 \times O(nd^2)$

Key insight: LOOCV shortcut and AIC/BIC have same computational cost as validation set!

Implementation Guidelines

Stratification: For classification, ensure each fold has similar class proportions

Randomization: Shuffle data before folding (unless time series)

Reproducibility: Set random seed for fold assignment

Refitting final model: After hyperparameter selection, refit on all training data before final test evaluation

Standard errors: CV provides standard error estimates across folds; AIC/BIC don’t

Model averaging: AIC supports model averaging (weighted by $\exp(-\frac{\text{AIC}}{2})$); BIC and CV typically select single model

Connection to Learning Theory: Structural Risk Minimization

We've seen three frameworks for model selection: empirical (CV), parametric (AIC/BIC), and VC theory (Week 2). How do these relate?

? Student Question

“Cross-validation is empirical, AIC/BIC are parametric, VC dimension is combinatorial. Are these just three unrelated toolboxes?”

They seem to come from completely different domains with different assumptions. Why would they be connected?

Three Frameworks, Three Domains

Framework	Domain	Tool	Assumptions
Empirical	Data-driven	Cross-validation	None (distribution-free)
Parametric	Information theory	AIC/BIC	Parametric MLE + regularity
Theoretical	Learning theory	VC dimension	None (PAC bounds)

These seem like three separate approaches:

- **CV:** Resample data, estimate error empirically
- **AIC/BIC:** Penalize likelihood by parameter count
- **VC dimension:** Combinatorial complexity measure from shattering

What could they possibly have in common?

The Fragmentation Problem

⚠ Crisis

The Fragmentation Problem

Without a unifying principle, we must answer separately for each framework:

For decision trees:

- CV: Run 5-fold, get error estimate
- AIC/BIC: What's the parameter count? Node count? Depth? Unclear!
- VC dimension: $O(n \log n)$ for depth- d trees

For ridge regression:

- CV: LOOCV shortcut available
- AIC: $k = d$ parameters (but what about effective df?)
- VC dimension: Infinite-dimensional (but effective dimension via eigenvalues?)

For neural networks:

- CV: Only option (no shortcuts)
- AIC/BIC: Regularity conditions questionable
- VC dimension: Depends on architecture, can be huge

Three separate analyses for every model class! No unified understanding.

The Common Goal: Test Error Estimation

All three frameworks solve the **same problem**: estimate how well the model generalizes to unseen data.

💡 Aha Moment

The Unification

All methods decompose generalization as:

$$\text{Generalization error} = \text{Training error} + \text{Complexity penalty} \quad (37)$$

The frameworks differ only in **how they define the penalty**:

- **AIC**: Add $2k$ for each parameter. Example: $-2 \log L(\hat{\theta}) + 2 \cdot 10 = -2 \log L + 20$
- **BIC**: Add $k \log n$ (grows with sample size). Example with $n = 100, k = 10$: $-2 \log L + 10 \log 100 \approx -2 \log L + 46$
- **SRM**: Add $C \sqrt{\frac{d \log n}{n}}$ for VC dimension d . Example with $d = 5, n = 100$: $\hat{R} + C \sqrt{\frac{5 \log 100}{100}} \approx \hat{R} + 0.21C$
- **CV**: Penalty is implicit—estimated empirically by averaging held-out errors

Notice: BIC penalizes complexity more heavily than AIC for large n , while SRM's penalty depends on capacity (VC dimension) rather than just parameter count.

Framework	Domain	Training Error	Penalty	Assump- tions	Best For
5-fold CV	Any model	Empirical average	Implicit (em- pirical)	None	Default choice, flexible
LOOCV	Any model	Leave-one- out average	Implicit (em- pirical)	None	Small n , linear models
AIC	Parametric MLE	$-2 \log L(\hat{\theta})$	$2k$	Model fam- ily speci- fied	Prediction, large n
BIC	Parametric MLE	$-2 \log L(\hat{\theta})$	$k \log n$	True model exists	Inference, consis- tency
SRM	Any hypothe- sis class	$\hat{R}(\text{ERM})$	$C \sqrt{\frac{d \log n}{n}}$	None	Theoretical guar- antees

Table 1: Framework unification: all methods balance training error and complexity

All select the model minimizing: **fit to data + complexity cost**.

Structural Risk Minimization

From Week 2, we have the VC bound: with probability $\geq 1 - \delta$,

$$R(h) \leq \hat{R}(h) + O\left(\sqrt{\frac{d \log\left(\frac{n}{d}\right) + \log\left(\frac{1}{\delta}\right)}{n}}\right) \quad (38)$$

where $d = \text{VCdim}(\mathcal{H})$.

For model selection across hypothesis classes $\mathcal{H}_1 \subset \mathcal{H}_2 \subset \dots \subset \mathcal{H}_K$:

Definition: Structural Risk Minimization (SRM)

Select the hypothesis class $\mathcal{H}_k$ minimizing:

$$\hat{R}(h_k) + C \sqrt{\frac{d_k \log n}{n}} \quad (39)$$

where:

- $h_k \in \mathcal{H}_k$ is the ERM solution in class k
- $d_k = \text{VCdim}(\mathcal{H}_k)$ is the VC dimension
- C is a constant (typically tuned via validation)

This balances training error (decreases with k) against complexity (increases with k).

Example: Polynomial Regression via SRM

Compare polynomial degrees $d = 1, 2, \dots, 10$ with $n = 100$ samples.

VC dimension: $\text{VCdim}(\text{polynomials of degree } d) = d + 1$

Degree d	Training Error	VC dim	Penalty	SRM Score
1	0.40	2	0.14	0.54
2	0.25	3	0.17	0.42
3	0.15	4	0.20	0.35
5	0.10	6	0.24	0.34
8	0.05	9	0.30	0.35
10	0.03	11	0.33	0.36

SRM selects: Degree 5 (minimum SRM score = 0.34)

The penalty prevented selecting degree 10 despite its lower training error.

Decision Guide for Model Selection

Which method should you use? It depends on your sample size, model type, and goals.

Sample Size	Model Type	Candidates	Recommended Method	Why
$n < 100$	Linear models	Few (< 10)	LOOCV with shortcut	Unbiased, efficient closed form
$n < 100$	Nonlinear	Few (< 10)	10-fold CV	Good bias-variance tradeoff
$n < 100$	Any	Many (> 50)	Validation set	Computational cost of CV too high
$100 \leq n < 1000$	Parametric (e.g., GLM)	Few-moderate	AIC or BIC	Fast, theoretically justified
$100 \leq n < 1000$	Nonlinear	Any	5-fold or 10-fold CV	Default robust choice
$100 \leq n < 1000$	Linear with many features	Regularization paths	LOOCV shortcut	Efficient for λ selection
$n \geq 1000$	Any	Few (< 20)	Validation set (80/20)	Simple, sufficient data
$n \geq 1000$	Neural networks	Many architectures	Validation + early stopping	Computational efficiency
$n \geq 1000$	Any	Need consistency	BIC	Selects true model as $n \rightarrow \infty$
Any n	Hypothesis classes	Theoretical comparison	SRM	Distribution-free PAC guarantees

Table 2: Practical decision guide for choosing model selection method

Key principles:

- **Small $n < 100$:** Maximize data use (LOOCV, k-fold CV)
- **Medium $100 \leq n < 1000$:** Balance accuracy and speed (CV, AIC/BIC)
- **Large $n \geq 1000$:** Simple validation often sufficient
- **Parametric models:** Consider AIC/BIC for speed and theory
- **Need consistency:** Use BIC (selects true model)
- **Need worst-case guarantees:** Use SRM (PAC bounds)

Penalty behavior for large n :

- **AIC** ($2k$): Constant—prediction-oriented
- **BIC** ($k \log n$): Growing—most conservative, consistency-oriented
- **SRM** ($C\sqrt{\frac{k \log n}{n}}$): Shrinking—VC bound becomes tight

Key Takeaway

The meta-lesson:

All model selection methods estimate: **Training error + Complexity penalty**

They differ in:

- How complexity is measured (parameters vs VC dimension vs empirical)
- What assumptions are required (none vs parametric vs regularity)
- What guarantees they provide (empirical vs average-case vs worst-case)

The “right” method depends on your model class and goals, but they’re all solving the same fundamental problem: **trading off fit to training data against model complexity**.

Progress on backbone: Model selection unifies empirical, parametric, and theoretical approaches. All balance training performance against complexity. This connection reveals that CV, AIC/BIC, and VC theory are different lenses on the same underlying principle: preventing overfitting by penalizing complexity.

Summary and Connections

Key Insights

1. **Cross-validation** estimates test error by simulating train/test splits
2. **LOOCV** is nearly unbiased but has efficient closed forms for linear models
3. **AIC** minimizes prediction error; derived from KL divergence
4. **BIC** selects true model consistently; derived from Bayesian principles
5. **MDL** formalizes Occam's razor via coding theory
6. **SRM** provides distribution-free guarantees via VC theory

Connection

To Week 4: Regularization parameter λ is selected via CV or information criteria.

To Week 6: Kernel selection in RKHS uses same principles—CV for hyperparameters, complexity penalties for model selection.

To Week 9-10: Boosting iterations as model complexity—when to stop? Use CV or validation set.

To Week 11-12: Neural architecture search and early stopping—modern applications of model selection.

Synthesis Challenge

The Model Selection Toolbox:

You've now learned five approaches to model selection. For each scenario below, recommend the best approach and justify your choice:

Scenario A: $n = 50$ samples, comparing polynomial degrees 1-5, linear regression.

- Options: Validation set, 5-fold CV, LOOCV, AIC, BIC

Scenario B: $n = 10,000$ samples, comparing 100 neural network architectures, nonlinear.

- Options: Validation set, 5-fold CV, 10-fold CV, early stopping

Scenario C: $n = 200$ samples, comparing 5 linear models, need to find true sparse features.

- Options: 10-fold CV, LOOCV, AIC, BIC, SRM

Scenario D: $n = 1,000$ samples, ridge regression with 50 candidate λ values.

- Options: LOOCV with shortcut, 5-fold CV, AIC, nested CV

For each:

1. What's your recommendation?
2. What are the tradeoffs?
3. What assumptions are you making?
4. How would you report the final performance?

Closing Challenge

Opening Challenge

Revisiting the Opening Question:

For ridge regression with $n = 100$, $d = 50$:

1. Validation set (70/30):

- Pros: Fast, simple
- Cons: Wastes 30 samples, high variance
- Use when: n is large, speed matters

2. LOOCV:

- Pros: Nearly unbiased, uses all data
- Cons: High variance, expensive (but $O(nd^2)$ with shortcut!)
- Use when: n is small, accuracy matters

3. AIC:

- Pros: Fastest ($O(nd^2)$), no refitting
- Cons: Assumes model family, asymptotic
- Use when: Model is well-specified, n is large

Recommendation: For $n = 100$, use LOOCV with closed form (fast and accurate) or 10-fold CV (good tradeoff). AIC is reasonable if Gaussian assumptions hold.

Confidence Check

Rate your mastery of Week 5 (1-5):

Cross-Validation:

- Can you explain the bias-variance tradeoff in k-fold CV?
- Can you derive the LOOCV shortcut for linear models?
- Can you implement nested CV to avoid selection bias?

Information Criteria:

- Can you derive AIC from KL divergence?
- Can you explain when BIC is consistent and when AIC is not?
- Can you compute AIC/BIC for Gaussian linear models?

Practical Skills:

- Can you choose the right selection method for a given scenario?
- Can you avoid common pitfalls like selection bias?
- Can you estimate computational costs for different approaches?

If < 4 on any: Review the corresponding sections, work through examples, and try the construct-your-own exercises.

Student Challenge

Exercise: Implement efficient LOOCV for ridge regression. Compare with naive refitting and verify the speedup. Then compare LOOCV, 10-fold CV, and AIC for selecting λ on a real dataset.

Research question: Modern deep learning uses millions of parameters but doesn't use traditional model selection criteria. Why do AIC/BIC fail for neural networks? What do we use instead? (Preview: Week 11 discusses implicit regularization and early stopping.)